

Shimano BT-E6000 BMS Protocol Analysis

Version 2.0 — Comprehensive Protocol Specification

Covers: UART frame format, CRC-16/IBM-SDLC algorithm, authentication protocol, telemetry decoding, register address map, cell-level BMS data, capabilities exchange, timestamp frames, session flow, and experimental validation.

Document	Shimano_BTE6000_BMS_Protocol_Analysis_V2.pdf
Date	2026-03-22
Hardware	BT-E6000 / BT-E6001 (36V 10S4P Li-Ion)
MCU	Renesas R2A240 60F020 (78K0 family)
BMS IC	TI BQ77PL900 (host mode)
Charger	EC-E6002 / EC-E6000
Data Sources	UART captures (RXTXRecords), 256-address register scan, replay experiments
CRC Algorithm	CRC-16/IBM-SDLC — verified 212/212 frames (100%)

Table of Contents

1. System Overview
 2. Physical Layer & Framing
 3. CRC-16/IBM-SDLC Algorithm
 4. SEQ Byte — Device Addressing
 5. Frame Types (TYPE Field)
 6. Authentication Protocol (TYPE 0x30)
 7. Capabilities Exchange (TYPE 0x31)
 8. Telemetry Frame (TYPE 0x10)
 9. Timestamp / Energy Frame (TYPE 0x32)
 10. Shutdown Command (TYPE 0x21)
 11. Complete Session Flows
 12. Register Address Map (256-Address Scan)
 13. Extended Register Analysis
 14. Experimental Validation (Discussion #4)
 15. Open Questions & Future Work
 16. Quick Reference Card
- Appendix A: Python Code Snippets
- Appendix B: Changelog V1 -> V2

1. System Overview

The Shimano STEPS BT-E6000 is a 36V e-bike battery pack using a 10S4P lithium-ion cell configuration. Internal communication between the battery and external devices (charger, motor controller, display unit) uses a proprietary serial UART protocol at 3.3V logic levels.

The battery contains two key ICs: a **Renesas R2A240 60F020** microcontroller (78K0 family, 8-bit) that handles all UART communication and protocol logic, and a **TI BQ77PL900** battery management IC that monitors cell voltages, temperature, and current in host mode. Experimental validation confirms the MCU operates independently of the BQ77PL900 — the full UART protocol (including auth, polling, and telemetry) functions even without a battery connected to the BMS board.

1.1 Communication Architecture

All external devices communicate with the battery MCU over a single-wire UART bus (no CAN). The protocol is half-duplex request/response: an external device sends a frame, and the battery MCU responds. The motor unit may act as a router distributing messages between nodes. Authentication is required before the battery enters active mode, but is replay-vulnerable.

Device	Greeting SEQ	Data SEQ	Role
Display (SC-E6010)	0x40	0x00-0x03	UI, assist mode control
Charger (EC-E6002)	0x41	0x00-0x03	Charge control, telemetry polling
Custom Tool	0x42	0x00-0x0F+	Register scan, diagnostics
Battery (BT-E6000)	0xC0-0xC2	0x80+N	Responds: SEQ = request_SEQ 0x80

Table 1: Device identifiers on the Shimano STEPS bus

2. Physical Layer & Framing

2.1 UART Parameters

The UART operates at **9600 baud**, **8N1** (8 data bits, no parity, 1 stop bit) with 3.3V logic levels. Frames are separated by a **0x00 delimiter byte** with typically ~1 second intervals between polling cycles.

2.2 Frame Structure

V2 Key Finding: The leading 0x00 byte is an inter-frame **delimiter**, not part of the frame payload. CRC-16/IBM-SDLC computes correctly over the frame bytes *excluding* the leading 0x00. All frame-type-dependent CRC init values from V1 are obsolete.

Field	Bytes	Description
Delimiter	1 (0x00)	Inter-frame separator — NOT included in CRC calculation
SEQ	1	Device address + sequence counter (see Section 4)
LEN	1	Number of bytes from TYPE to end of payload (inclusive of TYPE)
TYPE	1	Frame type identifier (0x10, 0x11, 0x12, 0x21, 0x30, 0x31, 0x32)
Payload	LEN-1	Type-specific payload data (0 bytes if LEN=1)
CRC	2	CRC-16/IBM-SDLC, little-endian, computed over SEQ..Payload

Table 2: Frame structure

LEN field semantics: LEN counts the TYPE byte plus all payload bytes. For greeting frames (LEN=0), no TYPE or payload exists — the frame is just [SEQ][0x00][CRC_L][CRC_H]. Experimental validation shows that LEN=1 (TYPE only, no payload) triggers an echo response for TYPE 0x10, while LEN >= 2 triggers a full telemetry response. Extra payload bytes beyond the expected length are ignored by the battery MCU.

2.3 Example Frames

Frame Type	Raw Bytes (with delimiter)	Breakdown
Greeting (Display)	00 40 00 21 49	[00] SEQ=40 LEN=00 CRC=2149
Greeting (Battery)	00 C1 00 35 DC	[00] SEQ=C1 LEN=00 CRC=35DC
Poll (charger)	00 00 05 10 00 00 00 00 B7 2C	[00] SEQ=00 LEN=05 TYPE=10 PL=00,00,00,00 CRC=B72C
Shutdown	00 02 01 21 27 5A	[00] SEQ=02 LEN=01 TYPE=21 CRC=275A
Timestamp	00 02 07 32 19 09 05 0D 1E 22 FC FD	[00] SEQ=02 LEN=07 TYPE=32 PL=6bytes CRC=FCFD

Table 3: Example frames

3. CRC-16/IBM-SDLC Algorithm

This is the single most important V2 finding. The CRC algorithm is standard **CRC-16/IBM-SDLC** (also known as CRC-16/ISO-HDLC, CRC-16/X.25), applied uniformly to ALL frame types. Independently confirmed by both analytical derivation and experimental testing (sending never-before-seen SEQ values and verifying the battery's response CRCs).

Parameter	Value
Polynomial	0x1021 (CRC-CCITT)
Init Value	0xFFFF
Reflect Input	True
Reflect Output	True
XOR Output	0xFFFF
Check Value	0x906E (for ASCII "123456789")
CRC Scope	All bytes from SEQ through end of Payload (excludes delimiter and CRC)
Byte Order	Little-endian (CRC low byte first)
Common Names	IBM-SDLC, ISO-HDLC, X.25, CRC-B

Table 4: CRC-16/IBM-SDLC parameters

3.1 Verification Results

Frame Type	Frames Tested	Result
Greeting (LEN=0)	3 + 64 from scan	67/67 OK
Auth / Shutdown (LEN=1)	4	4/4 OK
Poll (LEN=5, charger+bike)	10	10/10 OK
Timestamp (LEN=7)	7	7/7 OK
Capabilities (LEN=11, 18)	3	3/3 OK
Auth challenge (LEN=17)	4	4/4 OK
Auth response (LEN=18)	1	1/1 OK
Telemetry (LEN=22)	4 + 14 from replay	18/18 OK
Register scan responses	113 (data regs)	113/113 OK
Experimental (Discussion #4)	~20 novel frames	20/20 OK
TOTAL	~247	247/247 OK (100%)

Table 5: CRC verification across all sources

3.2 Experimental CRC Confirmation

Discussion #4 provided definitive proof: SEQ values 0x04 through 0x0F (never seen in original Shimano captures) were sent to the battery, and every response had a valid CRC-16/IBM-SDLC checksum. This eliminates any possibility of a lookup-table or special-case CRC implementation.

3.3 Python Implementation

```
import crcmod

crc_sdslc = crcmod.predefined.mkCrcFun('x-25') # IBM-SDLC

def compute_crc(frame_bytes_without_delimiter):
    crc = crc_sdslc(frame_bytes_without_delimiter)
    return crc.to_bytes(2, 'little')

def verify_frame(hex_with_delimiter: str) -> bool:
    raw = bytes.fromhex(hex_with_delimiter.replace(' ', ''))
```

```
frame = raw[1:]      # strip 0x00 delimiter
data = frame[:-2]    # everything except CRC
expected = frame[-2:] # last 2 bytes = CRC LE
return compute_crc(data) == expected
```

4. SEQ Byte — Device Addressing

The SEQ byte identifies the sending device and carries a sequence counter. The battery always responds with SEQ = request_SEQ | 0x80 (for range 0x00-0x7F). For range 0x80-0xBF, the battery echoes SEQ unchanged. Range 0xC0-0xFF produces a hardware loopback (TX=RX).

4.1 SEQ Ranges

Range	Bits 7-6	Response Rule	Function
0x00-0x3F	00	Reply SEQ = N 0x80 (LEN=2 data)	Data register read
0x40-0x7F	01	Reply SEQ = N 0x80 (LEN=0 greeting)	Bus enumeration / greeting
0x80-0xBF	10	Reply SEQ = N unchanged (LEN=2 data)	Extended register read
0xC0-0xFF	11	ECHO (TX = RX, no processing)	Hardware loopback, not processed

Table 6: SEQ byte range behavior

4.2 Sequence Counter

In normal Shimano operation, the low 2 bits of SEQ cycle 0-1-2-3 (mod 4). However, **experimental testing confirmed that SEQ values 0x04 through 0x0F all produce valid responses**. The mod-4 convention is a Shimano software choice, not a hardware limitation. The battery simply ORs 0x80 onto whatever SEQ value it receives.

TX SEQ	RX SEQ	Response	Source
0x00	0x80	LEN=2 data register	Register scan
0x04	0x84	LEN=22 full telemetry	Discussion #4 experiment
0x0F	0x8F	LEN=22 full telemetry	Discussion #4 experiment
0x40	0xC0	LEN=0 greeting	Normal display operation
0x41	0xC1	LEN=0 greeting	Normal charger operation
0x42	0xC2	LEN=0 greeting	Custom tool
0xA0	(none)	NO REPLY — consistent	Scan + Discussion #4
0xC0	0xC0	ECHO (TX=RX)	All sources

Table 7: SEQ response examples across all test sources

Address 0xA0: Consistently produces NO REPLY across both the fine-grained 256-scan and the coarse high-nibble scan. This is the only greeting-range address (0x40-0x7F mapped to 0xA0 at +0x60) that fails to respond. Also 0xAB echoes instead of responding with data.

5. Frame Types (TYPE Field)

TYPE	Name	LEN	Direction	Description
(none)	Greeting	0x00	Both	Device presence / bus enumeration
0x10	Poll / Telemetry	0x02-0x09 / 0x16	Req / Resp	Status poll (any LEN>=2 works) or telemetry response
0x11	Auth Acknowledge	0x01	Ext -> Batt	Authentication sub-frame A
0x12	Auth Commit	0x01	Ext -> Batt	Authentication sub-frame B
0x21	Shutdown	0x01	Ext -> Batt	Power-off / disconnect command
0x30	Auth Challenge	0x11 / 0x12	Both	Challenge (ext) or response (batt)
0x31	Capabilities	0x0B / 0x12	Both	Capability EXCHANGE (bidirectional!)
0x32	Timestamp	0x07	Ext -> Batt	RTC timestamp (YY MM DD HH MM SS)

Table 8: Known frame types

TYPE 0x10 minimum LEN: Experimental testing shows LEN=1 (TYPE only, no payload) produces an echo. LEN=2 and above triggers a full telemetry response (LEN=0x16). Extra payload bytes beyond the expected 4 are silently ignored by the battery MCU.

6. Authentication Protocol (TYPE 0x30)

Before entering active mode, the external device must complete an authentication handshake. This is a multi-frame sequence.

6.1 Authentication Sequence

Step	Direction	TYPE	Description
1	Ext -> Batt	0x30 (LEN=0x11)	Challenge: device ID (2B) + 12-byte LFSR nonce + 2B reserved
2	Ext -> Batt	0x11 (LEN=0x01)	Auth acknowledge (TYPE only, no payload)
3	Ext -> Batt	0x12 (LEN=0x01)	Auth commit (TYPE only, no payload)
4	Batt -> Ext	0x30 (LEN=0x12)	Response: device ID + 15-byte truncated MAC

Table 9: Authentication handshake

6.2 Challenge Structure

Offset	Len	Field	Description
0-1	2	Device ID	0x02,0x01 = Charger; 0x03,0x02 = Motor controller
2-3	2	Padding	Always 0x00 (bike) or nonce start (charger)
4-15	12	Challenge nonce	LFSR-generated pairs; bytes 4-7 repeat at bytes 12-15

6.3 Security Assessment

The authentication is **fully replay-vulnerable**. Experimental confirmation from Discussion #4: a replayed charger authentication sequence was accepted by the MCU even with **no battery connected** to the BMS board. No session counter, timestamp, or monotonic nonce is included. The response MAC is hypothesized to be HMAC-SHA-1 (per TI BQ77PL900 App Note SLUA625), truncated to 15 bytes. The sequence counter does not need to match — any SEQ value is accepted.

7. Capabilities Exchange (TYPE 0x31)

V2 Finding: TYPE 0x31 is a **bidirectional capability exchange**, not a unilateral battery announcement. The external device first sends its own capabilities (LEN=0x0B), and the battery responds with its capabilities (LEN=0x12). This was discovered through the replay experiments in Discussion #4.

7.1 Exchange Flow

External sends (LEN=0x0B = TYPE + 10 payload bytes):

```
00 03 0B 31 9F 01 A9 01 01 00 02 00 1F 00 05 7F
  ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^
  FW  Model  Cfg  Rate Param
```

Battery responds (LEN=0x12 = TYPE + 17 payload bytes):

```
00 83 12 31 25 9F 01 A9 01 01 00 05 00 28 00 6E 00 10 01 78 00 76 BD
  ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^ ^^
  Ctx FW  Model  Cfg  Rate VMax DynA DynB      Cap
```

7.2 Context Byte (0x25)

The battery response includes a **context byte 0x25** (decimal 37) as the first payload byte after TYPE. This same value appears in register 0xB7 (byte 1) and 0xB9 (byte 1), and in the telemetry response offset 0 during replay sessions. It may represent a firmware build number, session token, or MCU internal state identifier.

7.3 Capability Fields (uint16-LE)

Offset	Field	External	Battery (no batt)	Battery (charger)	Battery (scan)
0-1	FW Version	0x019F=415	0x019F=415	0x019F=415	0x019F=415
2-3	Model ID	0x01A9=425	0x01A9=425	0x01A9=425	0x01A9=425
4-5	Config	0x0001	0x0001	0x0001	0x0001
6-7	Rate	0x0002	0x0005	0x0005	0x0005
8-9	Max Voltage	0x001F=31	0x0028=40	0x0028=40	0x0028=40
10-11	Dynamic A	—	0x006E=110	0x0088=136	0x006E=110
12-13	Dynamic B	—	0x0110=272	0x0143=323	0x010E=270
14-15	Capacity	—	0x0078=120	0x0078=120	0x0078=120

Table 10: Capability field comparison across all contexts

Dynamic fields: Offsets 10-13 change depending on battery presence and state. DynA: 110 (no battery / scan) vs 136 (charger with battery). DynB: 272 (no battery) vs 270 (scan) vs 323 (charger with battery). These likely reflect state-of-health, remaining capacity, or accumulated energy.

8. Telemetry Frame (TYPE 0x10)

8.1 Poll Frame (External -> Battery, LEN >= 0x02)

Offset	Field	Values	Description
0	MODE	0x02=Init, 0x03=Active	State; transitions 0x02->0x03 at startup
1	CONTEXT	0x00=Charger, 0x03=Startup, 0x09=Bike, 0x0D=Other	Device mode context
2-3	Reserved	0x00, 0x00	Always zero; extra bytes beyond offset 3 are ignored

Table 11: Poll frame payload (TYPE 0x10)

8.2 Telemetry Response (Battery -> External, LEN=0x16)

The battery responds with 22 bytes of telemetry. The no-battery experiment (Discussion #4) was crucial for identifying which bytes are board-level sensors vs BQ77PL900 data.

Off	Len	Field	No Battery	With Battery	Interpretation
0	1	Context ID	0x25=37	0x00	Session/device context byte
1	1	State byte	0x00	0x02 / 0x03	Related to MODE in poll frame
2-4	3	Sub-state	0x02,0x02,0x00	0x00,0x00,0x00	Internal MCU state
5-6	2	Pack Voltage	0x00F8=248mV	0x9FFD=40957mV	uint16-LE, millivolts
7	1	Current A	0x68=104	0x08(pre)/0x68(CC)	Current measurement channel A
8	1	Unknown	0x00	0x20	Constant 0x20 when battery present
9	1	Current B	0x08	0xFA(pre)/0x4A(CC)	Current measurement channel B
10	1	Pack Temp	0x00	0x1F=31 / 0x20=32	Pack temperature (Celsius)
11	1	Board Temp	0x15=21	0x18=24	MCU board ambient sensor (Celsius)
12	1	BMS Temp	0xE3=227(!)	0x18=24	BQ77PL900 sensor; 227=DISCONNECTED
13	1	Board Temp 2	0x15=21	0x18=24	Second board ambient sensor
14	1	Status A	0x4A=74	0x5C=92	BMS status / accumulated value
15	1	Status B	0x00	0x00/0x4D/0x4E	0x00=idle, 0x4D/4E=charging
16-21	6	Reserved	all 0x00	all 0x00	Unused

Table 12: Telemetry response — with and without battery comparison

Critical finding from no-battery test: Offset 12 reads 0xE3 (227) when the BQ77PL900 has no battery connected, but 24 C with a battery. This definitively identifies offset 12 as the **BQ77PL900 internal temperature sensor**, with 227 as a disconnect/error sentinel. Offsets 11 and 13 read ambient board temperature (21 C) regardless of battery presence.

8.3 Charge Session Progression

Frame	Voltage	I_raw_A	I_raw_B	TempPack	Board	BMS	Board2	StA	StB	
1	40.957V	0x08	0xFA	31 C	24	24	24	0x5C	0x00	(pre-charge)
3	41.371V	0x68	0x4A	32 C	24	24	24	0x5C	0x4E	(CC start)
15	41.489V	0x80	0x62	32 C	24	24	24	0x5D	0x4D	
31	41.542V	0x8A	0x6A	32 C	24	24	25	0x5D	0x4E	(late CV)

9. Timestamp / Energy Frame (TYPE 0x32)

TYPE 0x32 carries a 6-byte RTC timestamp, sent periodically (~1/min) during active sessions.

Offset	Field	Format	Example
0	Year	Hex, add 2000	0x19 = 2025
1	Month	Hex 01-12	0x09 = September
2	Day	Hex 01-31	0x05 = 5th
3	Hour	Hex 00-23	0x0D = 13 (1 PM)
4	Minute	Hex 00-59	0x1E = 30
5	Second	Hex 00-59	0x22 = 34

Table 13: Timestamp frame payload

Bike sessions (consistent minute-by-minute progression):

ON_OFF_S1:	2025-09-05 13:30:34	TIMEOUT_1:	2025-09-05 14:07:00
ON_OFF_S2:	2025-09-05 13:45:07	TIMEOUT_12:	2025-09-05 14:13:00
MENU:	2025-09-05 13:59:00	ONLINE:	2000-01-12 18:08:32 (RTC not set)

10. Shutdown Command (TYPE 0x21)

TYPE 0x21 is a 1-byte frame (LEN=0x01, TYPE only) signaling power-off. The preceding poll frame shows MODE reverting from 0x03 to 0x02.

```
Pre-shutdown poll (MODE reverts): 00 01 05 10 02 09 00 00 0A 16
Shutdown command:                  00 02 01 21 27 5A
Final delimiter:                   00
```

Observed in both manual power-off and auto-timeout (~5 min idle).

11. Complete Session Flows

11.1 Universal Startup Sequence

Step	SEQ	Frame	Description
1	0x4N	[SEQ] 00 [CRC]	External greeting (LEN=0)
	0xCN	[SEQ] 00 [CRC]	Battery greeting response
2	0x01	[SEQ] 11 30 [challenge] [CRC]	Auth challenge (TYPE 0x30, LEN=0x11)
3	0x02	[SEQ] 01 11 [CRC]	Auth acknowledge (TYPE 0x11)
4	0x03	[SEQ] 01 12 [CRC]	Auth commit (TYPE 0x12)
4b	0x03	[SEQ] 0B 31 [caps] [CRC]	Capabilities exchange (TYPE 0x31, LEN=0x0B)
	0x83	[SEQ] 12 31 [batt_caps] [CRC]	Battery capabilities response (LEN=0x12)
5	0x00	[SEQ] 05 10 02 03 00 00 [CRC]	First poll: MODE=0x02 (init), CTX=0x03
6	0x01	[SEQ] 05 10 03 03 00 00 [CRC]	Second poll: MODE=0x03 (active)
7	0x02	[SEQ] 07 32 [timestamp] [CRC]	Timestamp frame (TYPE 0x32)
8+	cycle	[SEQ] 05 10 03 [ctx] 00 00 [CRC]	Steady-state (CTX=0x09 bike, 0x00 charger)

Table 14: Universal startup sequence (updated with capabilities exchange)

The capabilities exchange (step 4b) occurs between auth commit and the first poll. This was not visible in earlier unidirectional captures that only recorded battery responses.

11.2 Session Lifecycle

```

Charger:  Greeting -> Auth -> Caps Exchange -> Poll(MODE=02) -> Poll(MODE=03)
          -> Timestamp -> Steady-state CC/CV charging -> MODE reverts to 02 -> Shutdown

Bike:     Greeting -> Auth -> Caps Exchange -> Poll(MODE=02) -> Poll(MODE=03)
          -> Timestamp -> Steady-state (CTX=09) -> Idle timeout -> Shutdown

Replay:   Greeting(41) -> Auth replay(30+11+12) -> Caps(31) -> Poll(10)
          -> Full telemetry responses (works even without battery!)

```

12. Register Address Map (256-Address Scan)

A complete 256-address scan was performed using a custom tool with SEQ=0x42. All 177 response CRCs verified 100% with IBM-SDLC. Scan date: 2026-03-21.

12.1 Address Space Overview

Range	Count	Response Type	Description
0x00 - 0x3F	50 data + 14 NR	LEN=2 (+ 1 extended)	Data registers; all Data[1]=0x01
0x40 - 0x7F	64 greeting	LEN=0	Bus enumeration; battery responds SEQ 0x80
0x80 - 0xBF	53 data + 5 ext + 5 NR	Mixed	Extended registers at 0x92, 0x97, 0xA9, 0xB7, 0xB9
0xC0 - 0xFF	64 echo	TX = RX	Hardware loopback; not processed by MCU

Table 15: Address space overview

12.2 Standard Registers (LEN=2)

107 addresses return LEN=2 with Data[0] = CRC low byte of the request frame and Data[1] = 0x01 uniformly. This appears to be an echo/acknowledge mechanism. As confirmed in Discussion #4, these registers "use the first CRC byte from the message in the reply."

12.3 NO_REPLY Addresses

Low range (0x00-0x3F): 0x13, 0x14, 0x19, 0x1D, 0x21, 0x25, 0x26, 0x28, 0x2F
 High range (0x80-0xBF): 0x95, 0x9B, 0x9C, 0xA0, 0xA7
 Anomaly: 0xAB returns ECHO instead of data (only non-0xC0+ echo)

Address 0xA0 consistently produces NO_REPLY across both the fine 256-scan and the coarse high-nibble scan from Discussion #4. May require prior authentication or be unimplemented.

13. Extended Register Analysis

Six addresses return extended responses (LEN > 2) containing multi-byte BMS data blocks.

13.1 Register 0x1A — Serial / ID Block (LEN=7)

Data: A6 A1 17 60 56 52 4B

7-byte block, likely a binary-encoded production/serial identifier.

13.2 Register 0x92 — Status (LEN=3)

Data: AA 90 01

Byte 1 = 0x90 may encode BMS state flags. Byte 2 = 0x01 = standard ack.

13.3 Register 0x97 — BMS Configuration (LEN=10)

Data: 12 25 00 E1 00 00 90 10 0F 00

Offset	uint16-LE	Hex	Possible Meaning
0-1	9490	0x2512	BQ77PL900 config / FW build
2-3	57600	0xE100	BMS parameter (OV threshold?)
4-5	0	0x0000	Reserved
6-7	4240	0x1090	BMS parameter (UV threshold?)
8-9	15	0x000F	Cell count bitmask?

Table 16: Register 0x97 fields (tentative)

13.4 Register 0xA9 — BMS Mega-Block (LEN=43)

The **most data-rich register** — 43 bytes of BMS telemetry including pack voltage, capacity, temperature, and current.

Raw: A0 00 00 00 00 1F 00 00 00 34 00 34 00 20 00 17 00 0D 00 08
00 04 00 0C 00 1F 20 F8 2A 4A 00 C7 00 D4 FE C4 00 01 00
00 00 02 00

Offset	uint16-LE	Interpretation	Conf.
0	0x00A0=160	Header / register echo byte	Med
1-4	all zeros	Reserved / padding	Low
5-6	0x001F=31	Unknown parameter	Low
7-24	see text	10x uint8 padded: 0,52,52,32,23,13,8,4,12,31	Med
25-26	0x201F=8223	Pack voltage: 8223/200 = 41.115V	HIGH
27-28	0x2AF8=11000	Remaining capacity: ~11.0 Ah	HIGH
29-30	0x004A=74	Temperature: 74-40 = 34 C	HIGH
31-32	0x00C7=199	Unknown (cycle count? SOH?)	Low
33-34	0xFED4=-300	Current: -300*10mA = -3.0A (charging)	HIGH
35-36	0x00C4=196	Unknown (accumulated charge?)	Low
37-42	various	BMS state flags and reserved	Low

Table 17: Register 0xA9 mega-block fields

Voltage validation: 0xA9 offset 25-26 gives 41.115V (via val/200), matching the concurrent telemetry readings of 41.0-41.5V. Capacity of 11000 (11.0 Ah) aligns with the BT-E6000 nominal 11.6 Ah (418 Wh / 36V).

13.5 Register 0xB7 — Version/Status (LEN=3)

Data: 21 25 FF

Byte 0 = 0x21 (TYPE echo?). Byte 1 = 0x25 = context byte (matches capabilities and telemetry context). Byte 2 = 0xFF (all features enabled / hardware revision bitmask).

13.6 Register 0xB9 — Capabilities Mirror (LEN=18)

Returns capabilities prefixed with [0x31 (TYPE echo), 0x25 (context)]. Payload matches charger capabilities at 14/16 bytes. See Table 10 for full comparison.

14. Experimental Validation (Discussion #4)

GitHub Discussion #4 (michieltvg, 2026-03-20) documents a series of experiments performed on the BMS PCB **without a battery connected**. These experiments confirmed and extended multiple protocol findings.

14.1 Replay Attack Confirmation

A captured charger authentication sequence was replayed successfully. The MCU accepted the replayed auth, returned capabilities, and provided full telemetry responses. This proves: (a) auth is replay-vulnerable, (b) the MCU operates independently of the BQ77PL900, (c) the sequence counter value is not validated against a previous session.

14.2 Sequence Counter Range

SEQ values 0x00 through 0x0F were tested in poll frames. All produced valid telemetry responses with correct CRCs. The battery simply ORs 0x80 onto the received SEQ. The mod-4 wrapping (0-3) seen in Shimano firmware is a software convention only.

14.3 No-Battery Telemetry Baseline

Telemetry from the MCU without battery provides a crucial reference baseline:

Field	No Battery	With Battery	Significance
Voltage (off 5-6)	248 mV	40957 mV	Near-zero confirms voltage path
Current A (off 7)	0x68=104	0x08=8 (pre-chg)	0x68 = idle/baseline current reading
Board temp (off 11)	21 C	24 C	Ambient temp; slight warm-up with battery
BMS temp (off 12)	227 (!)	24 C	227 = BQ77PL900 DISCONNECTED sentinel
Board temp 2 (off 13)	21 C	24 C	Second board sensor, matches offset 11
Context (off 0)	0x25	0x00	Session context byte, not a status flag

Table 18: No-battery vs with-battery telemetry comparison

14.4 Poll LEN Experiment

TYPE 0x10 was sent with varying LEN values (1 through 9). Results:

LEN	Payload Size	Response	Interpretation
1	0 (TYPE only)	ECHO (TX=RX)	Not recognized as valid poll
2	1 byte	Full telemetry (LEN=0x16)	Minimum valid poll
3-9	2-8 bytes	Full telemetry (LEN=0x16)	Extra bytes ignored

Table 19: Poll LEN experiment results

14.5 High-Nibble Scan (Coarse)

Before the full 256-scan, a coarse scan with SEQ = 0x00, 0x10, 0x20, ..., 0xF0 was performed. This confirmed the four address zones (data / greeting / data / echo) and identified the consistent NO_REPLY at 0xA0.

15. Open Questions & Future Work

#	Question	Pri	Approach
1	Current scaling (telemetry offsets 7 and 9)	HIGH	In-line shunt measurement during CC/CV; baseline 0x68 from no-batt test helps
2	Register 0xA9 bytes 7-24 (cell-level data?)	HIGH	Compare with individual cell voltages; BQ77PL900 I2C tap
3	Context byte 0x25 meaning	MED	Test with different FW versions; may be build number
4	Capabilities dynamic fields (DynA, DynB)	MED	Compare across batteries with different SOH / cycle counts
5	Auth key extraction from R2A240 flash	MED	UART bootloader (FLMD0 pin), E2 Lite, or voltage glitching
6	Register scan after authentication	MED	NO_REPLY addresses may respond post-auth; new extended blocks?
7	0xA9 offset 31-32 (value 199)	LOW	Correlate across batteries of different ages
8	BQ77PL900 I2C bus sniffing	LOW	Ground truth for cell voltages, current, and temp readings

Table 20: Open questions

16. Quick Reference Card

Frame Format

```
[0x00 delim][SEQ][LEN][TYPE + Payload (LEN bytes)][CRC_L][CRC_H]
```

CRC: IBM-SDLC over [SEQ..Payload] (init=FFFF, poly=1021, ref=T/T, xor=FFFF)

Device SEQ Values

Display greeting:	0x40	Battery response:	0xC0
Charger greeting:	0x41	Battery response:	0xC1
Custom greeting:	0x42	Battery response:	0xC2
Data frames:	0x00-0x0F+	Battery:	SEQ 0x80
Range 0xC0-0xFF:	ECHO (loopback, not processed)		

Startup Sequence

1. Greeting	->	Battery greeting response
2. Auth 0x30	->	(queued)
3. Auth 0x11	->	(queued)
4. Auth 0x12	->	Battery auth response 0x30
5. Caps 0x31	->	Battery capabilities 0x31
6. Poll MODE=0x02	->	Telemetry (init)
7. Poll MODE=0x03	->	Telemetry (active)
8. Timestamp 0x32	->	Steady-state polling

Key Registers

0x1A: Serial/ID (7B)	0xB7: Version (3B: [0x21, 0x25, 0xFF])
0x92: Status (3B)	0xB9: Capabilities mirror (18B)
0x97: BMS config (10B)	0xA9: BMS MEGA-BLOCK (43B):

0xA9 Key Fields

Off 25-26: Pack voltage	(uint16-LE / 200 = Volts)
Off 27-28: Capacity	(uint16-LE, ~10 mAh units)
Off 29-30: Temperature	(uint16-LE - 40 = Celsius)
Off 33-34: Current	(int16-LE, ~10 mA, negative = charging)

Appendix A: Python Code Snippets

A.1 CRC Calculation & Verification

```
import crcmod
crc_sdlc = crcmod.predefined.mkCrcFun('x-25')

def verify_frame(hex_with_delimiter: str) -> bool:
    raw = bytes.fromhex(hex_with_delimiter.replace(' ', ''))
    frame = raw[1:]          # strip delimiter
    data = frame[:-2]        # everything except CRC
    expected = frame[-2:]    # CRC LE
    computed = crc_sdlc(data).to_bytes(2, 'little')
    return computed == expected
```

A.2 Build Greeting & Poll Frames

```
def build_greeting(device_id: int) -> bytes:
    data = bytes([device_id, 0x00])
    crc = crc_sdlc(data).to_bytes(2, 'little')
    return b'\x00' + data + crc

def build_poll(seq: int, mode=0x00, ctx=0x00) -> bytes:
    data = bytes([seq, 0x05, 0x10, mode, ctx, 0x00, 0x00])
    crc = crc_sdlc(data).to_bytes(2, 'little')
    return b'\x00' + data + crc
```

A.3 Parse Telemetry Response

```
def parse_telemetry(hex_frame: str) -> dict:
    raw = bytes.fromhex(hex_frame.replace(' ', ''))
    p = raw[4:-2] # skip delim+SEQ+LEN+TYPE and CRC
    return {
        'context': p[0],
        'mode': p[1],
        'voltage_V': int.from_bytes(p[5:7], 'little') / 1000,
        'i_raw_a': p[7],
        'i_raw_b': p[9],
        'temp_pack': p[10],
        'temp_board': p[11],
        'temp_bms': p[12], # 227 = BQ77PL900 disconnected!
        'temp_board2': p[13],
        'status_a': p[14],
        'status_b': p[15],
    }
```

A.4 Parse Register 0xA9

```
import struct
def parse_reg_a9(data: bytes) -> dict:
    return {
        'pack_voltage_V': struct.unpack_from('f', data, 0),
        'capacity_Ah': struct.unpack_from('f', data, 4),
        'temperatur
e_C': struct.unpack_from('f', data, 8),
        'current_A': struct.unpack_from('f', data, 12)}
    }
```

Appendix B: Changelog V1 -> V2

Item	V1	V2	Impact
CRC Algorithm	Multiple init values per frame type	Single CRC-16/IBM-SDLC, 247+ frames verified	BREAKING
Leading 0x00	Part of frame (start byte)	Inter-frame delimiter (not CRC'd)	BREAKING
LEN field	Excludes TYPE	Includes TYPE	BREAKING
SEQ counter	2-bit (mod 4)	At least 4-bit (0-F confirmed); convention only	ENHANCED
SEQ response	Bit 7 = device flag	Full range mapping: +0x80, greeting, echo zone	ENHANCED
TYPE 0x31	Battery announcement only	Bidirectional capability EXCHANGE	BREAKING
TYPE 0x32	Not documented	Timestamp frame (YY MM DD HH MM SS)	NEW
TYPE 0x21	Not documented	Shutdown command	NEW
Telemetry off 0	Flag byte	Context/session ID (0x25 in replay, 0x00 in command)	ENHANCED
Telemetry off 12	Temp sensor 2	BQ77PL900 temp (227 = disconnected sentinel)	ENHANCED
Poll min LEN	Always LEN=5	LEN>=2 triggers telemetry; LEN=1 echoes	NEW
Register scan	Not available	Full 256-address map, 6 extended registers	NEW
0xA9 mega-block	Not available	43B BMS data: voltage, capacity, temp, current	NEW
No-battery test	Not available	MCU operates independently of BQ77PL900	NEW
Replay proof	Hypothesized	Confirmed experimentally, even without battery	ENHANCED

Table 21: Complete changelog V1 to V2

End of Document — Shimano BT-E6000 BMS Protocol Analysis V2

Generated 2026-03-22. Sources: UART captures, register scan, replay experiments, GitHub Discussion #4.